

SUPERVISED LEARNING · CLASSIFICATION

수학 없이 이해하는 머신러닝 분류

명확한 사례로 배우는 로지스틱 회귀, 평가 지표, KNN,
의사결정나무, 랜덤포레스트, 나이브 베이즈와 SVM

학습 대화와 regression.ipynb 기반 상세 정리
분류 파트 전체 및 보강 내용

이 문서의 목표와 범위

공식 암기 없이 “어떤 정보를 보고, 어떤 기준으로, 어느 클래스를 선택하는가”를 이해합니다.

최종 목표

분류는 입력 조건 x 를 보고 결과 y 가 어떤 레이블에 속하는지 예측하는 지도학습 방법이라고 자신의 말로 설명할 수 있어야 합니다.

목차

01	분류의 기본 구조, 이진·다중 분류
02	가능성, 임계값, 결정 경계
03	로지스틱 회귀와 reshape
04	정확도, 혼동 행렬, 정밀도·재현율·F1
05	KNN, 거리, 표준화, 붓꽃 예제
06	의사결정나무와 stratify
07	랜덤포레스트, GridSearchCV, 특성 중요도
08	나이브 베이즈와 TF-IDF 스팸 분류
09	SVM과 모델 선택
10	전체 실습 흐름, 노트북 점검 및 복습

읽는 방법

각 장의 예시를 먼저 읽고, 코드에서 그 예시가 어느 줄에 대응하는지 확인합니다. 수학 기호보다 데이터의 역할과 처리 순서를 우선합니다.

회귀와 분류의 차이

같은 입력 데이터를 사용해도 최종 질문이 무엇인지에 따라 회귀 또는 분류가 됩니다.

질문	모델의 답 y	문제
시험 점수는 몇 점인가?	83점	회귀
시험에 합격하는가?	합격	분류

학생 합격 예시

공부 시간 x	실제 정답 y
1시간	불합격
2시간	불합격
4시간	합격
5시간	합격

컴퓨터에서는 레이블을 숫자로 표현할 수 있습니다.

불합격 = 0
합격 = 1

숫자 레이블 주의

0과 1은 점수나 수량이 아니라 이름표입니다. 1이 0보다 더 큰 값을 예측한다는 의미가 아니라 서로 다른 두 클래스를 구분합니다.

입력 x

→

구분 규칙 학습

→

새 입력

→

레이블 y

한 문장 핵심

분류는 숫자의 크기가 아니라 새로운 데이터가 어느 종류에 속하는지 예측합니다.

이진 분류와 다중 분류

이진 분류

두 종류 중 하나를 선택합니다.

- 합격 / 불합격
- 스팸 / 정상
- 질병 있음 / 없음
- 사과 / 사과 아님

다중 분류

세 종류 이상 중 하나를 선택합니다.

- Setosa
- Versicolor
- Virginica

붓꽃 데이터

꽃받침 길이	꽃받침 너비	꽃잎 길이	꽃잎 너비	y
5.1	3.5	1.4	0.2	0: Setosa
6.0	2.9	4.5	1.5	1: Versicolor
6.5	3.0	5.8	2.2	2: Virginica

레이블 2가 레이블 0보다 우수하거나 두 배라는 의미는 없습니다. 클래스 이름을 숫자로 치환했을 뿐입니다.

사진 분류에서 X와 y

- **x**: 실제 사진에서는 픽셀의 색상 숫자 배열. 사람이 미리 만든 시스템이라면 빨간색, 둥근 정도 같은 특징일 수도 있습니다.
- **y**: 이진 분류라면 사과 1 / 사과 아님 0, 다중 분류라면 사과·배·바나나 중 하나입니다.

판단 기준

분류의 종류는 입력 x가 아니라 **최종적으로 선택해야 하는 레이블의 개수**로 구분합니다.

가능성과 최종 레이블은 다르다

많은 분류 모델은 최종 선택 전에 각 클래스에 대한 점수나 가능성을 계산합니다.

단계	학생 합격 예시
실제 정답 y	합격 1
모델의 중간 판단	합격 가능성 0.80, 불합격 가능성 0.20
최종 예측	합격 1

임계값

이진 분류에서는 흔히 50%를 기본 기준으로 사용합니다.

합격 가능성	50% 기준의 최종 예측
20%	불합격
48%	불합격
65%	합격
90%	합격

임계값을 낮춤

더 많은 사례를 양성 1로 판단합니다. 실제 양성을 덜 놓치지만 잘못된 경고가 늘 수 있습니다.

임계값을 높임

더 확실한 사례만 양성 1로 판단합니다. 잘못된 경고는 줄지만 실제 양성을 놓칠 수 있습니다.

상황별 선택

암 검진은 환자를 놓치지 않기 위해 임계값을 낮출 수 있고, 스팸 필터는 정상 업무 메일을 보호하기 위해 임계값을 높일 수 있습니다.

중요한 보정

모든 분류 모델이 확률을 직접 계산하는 것은 아닙니다. 기본 SVM은 경계와의 거리를 사용합니다. 분류의 본질은 확률 그 자체가 아니라 **최종 클래스를 선택하는 결정 규칙**입니다.

로지스틱 회귀는 분류 모델

이름에는 회귀가 있지만, 입력이 특정 클래스에 속할 가능성을 구해 레이블을 선택합니다.

왜 선형회귀를 그대로 쓰지 않을까?

선형회귀로 합격 여부를 예측하면 0시간에 -0.2, 10시간에 1.5처럼 확률로 해석할 수 없는 값이 나올 수 있습니다. 확률은 0%와 100% 사이여야 합니다.

로지스틱 회귀는 내부 점수를 S자 형태로 눌러 0과 1 사이의 값으로 만듭니다.

공부 시간	합격 가능성 예시	최종 판단
1시간	5%	불합격
3시간	25%	불합격
5시간	75%	합격
7시간	95%	합격

학습과 예측 코드

```
classifier = LogisticRegression()
classifier.fit(X_train, y_train)

classifier.classes_          # 모델이 아는 클래스 [0, 1]
classifier.predict([[6]])    # 최종 레이블
classifier.predict_proba([[6]]) # 클래스별 가능성
```

predict_proba 읽기

classes_가 [0, 1]이고 결과가 [[0.15, 0.85]]이면 불합격 0은 15%, 합격 1은 85%입니다. 최종 예측은 합격 1입니다.

결정 경계

S자 곡선이 50% 임계값과 만나는 공부 시간이 불합격과 합격 판단이 바뀌는 지점입니다.

reshape: 값은 그대로, 모양만 변경

모델과 그래프 함수가 요구하는 행·열 모양에 맞추기 위해 배열을 재배치합니다.

```
p = np.array([[1, 2, 3], [4, 5, 6]])
# shape: (2, 3)
```

코드	결과 모양	직관적인 의미
<code>p.reshape(-1)</code>	(6,)	[1, 2, 3, 4, 5, 6] 한 줄 목록
<code>p.reshape(-1, 2)</code>	(3, 2)	열 2개, 행 수는 자동 계산
<code>p.reshape(3, -1)</code>	(3, 2)	행 3개, 열 수는 자동 계산
<code>X.reshape(-1, 1)</code>	(데이터 수, 1)	특성 한 개를 가진 입력 표

-1은 음수 크기가 아니라 전체 값의 수를 보고 해당 위치의 크기를 자동 계산하라는 뜻입니다.

가능성 그래프에서 사용

```
X_range = np.arange(min(X), max(X), 0.1)
p = p.reshape(-1)

plt.plot(X_range, p, color="green")
plt.plot(X_range, np.full(len(X_range), 0.5), color="red")
```

- 초록 곡선: 공부 시간별 합격 가능성
- 빨간 가로선: 50% 임계값
- 파란 점: 실제 합격·불합격 데이터

노트북 수정 필요

셀 50의 주석은 `reshape(3, -1)`이지만 실제 코드는 `reshape(-1, 2)`를 반복합니다. 결과 모양은 이 예제에서 같지만, 의도에 맞추려면 코드를 `p.reshape(3, -1)`로 바꿔야 합니다.

정확도와 불균형 데이터

정확도는 전체 예측 중 맞힌 비율이지만, 이것만으로 모델을 평가하면 위험할 수 있습니다.

전체 사례	맞힌 사례	정확도
학생 10명	8명	80%, 즉 0.8

```
accuracy_score(y_test, y_pred)
classifier.score(X_test, y_test) # 로지스틱 회귀의 기본 score
```

정확도 99%인데 쓸모없는 모델

1,000명 중 정상 990명, 환자 10명인 데이터에서 모든 사람을 정상으로 예측합니다.

실제	모델 예측	결과
정상 990명	모두 정상	990명 맞음
환자 10명	모두 정상	10명 모두 놓침

정확도는 99%지만 환자를 한 명도 찾지 못합니다. 한 클래스가 지나치게 많은 상태를 **클래스 불균형**이라고 합니다.

stratify

`train_test_split(..., stratify=y)`는 원본의 클래스 비율을 학습·시험 데이터에도 비슷하게 유지합니다. 특히 클래스 수가 적거나 불균형할 때 유용합니다. 단, 불균형 자체를 해결하는 기능은 아닙니다.

한 문장 핵심

정확도는 출발점일 뿐입니다. 어떤 클래스를 어떤 방향으로 틀렸는지 혼동 행렬과 클래스별 지표로 확인해야 합니다.

혼동 행렬: 어떤 방식으로 틀렸는가

질병 있음 1을 양성, 질병 없음 0을 음성으로 두고 네 가지 결과를 구분합니다.

실제 상태	정상으로 예측	환자로 예측
실제 정상	TN 80명 정확한 음성	FP 5명 잘못된 경고
실제 환자	FN 3명 환자를 놓침	TP 12명 환자를 발견

이름 읽는 법

- **True:** 예측이 맞음, **False:** 예측이 틀림
- **Positive:** 양성 1이라고 예측, **Negative:** 음성 0이라고 예측
- **False Negative:** 음성이라고 예측했지만 틀렸으므로 실제로는 양성

다중 클래스 혼동 행렬

붓꽃처럼 클래스가 3개라면 행과 열이 각각 Setosa, Versicolor, Virginica인 3×3 표가 됩니다.

- 대각선 칸: 실제와 예측 클래스가 같으므로 정답
- 대각선 밖 칸: 어느 품종을 다른 품종으로 혼동했는지 표시

```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
             xticklabels=target_names,
             yticklabels=target_names)
```

한 문장 핵심

혼동 행렬은 “몇 개 맞혔나”를 넘어 무엇을 무엇으로 잘못 판단했는지 보여줍니다.

정밀도와 재현율

같은 혼동 행렬을 서로 다른 질문으로 읽습니다.

정밀도

환자라고 한 판단을 얼마나 믿을 수 있는가?

환자라고 예측한 17명 중 실제 환자가 12명이면 약 71%입니다.

정상인을 환자라고 잘못 경고하는 FP를 줄이는 데 관심이 있습니다.

재현율

실제 환자를 얼마나 많이 찾아냈는가?

실제 환자 15명 중 12명을 발견하면 80%입니다.

환자를 정상이라고 놓치는 FN을 줄이는 데 관심이 있습니다.

상황	특히 중요한 지표	이유
스팸 필터	정밀도	정상 업무 메일을 스팸으로 보내지 않도록 주의
암 선별 검사	재현율	실제 환자를 가능한 한 놓치지 않도록 주의

임계값과의 관계

- 임계값을 높이면 확실한 사례만 양성으로 판단하여 정밀도가 오르는 경향이 있지만 재현율은 낮아질 수 있습니다.
- 임계값을 낮추면 양성을 더 많이 찾아 재현율이 오르는 경향이 있지만 잘못된 경고가 늘 수 있습니다.

정답은 상황에 따라 다름

정밀도와 재현율 중 항상 하나가 더 중요하지는 않습니다. 잘못된 경고와 놓침 중 어떤 실수가 더 위험한지 먼저 정해야 합니다.

F1 점수와 분류 보고서

F1은 정밀도와 재현율이 모두 괜찮은지 하나의 점수로 요약합니다.

정밀도	재현율	F1의 상태
90%	90%	높음
90%	30%	낮은 쪽의 영향으로 낮아짐
40%	95%	낮은 쪽의 영향으로 낮아짐
70%	75%	비교적 균형적

```
print(classification_report(
    y_test, y_pred,
    target_names=iris.target_names
))
```

어떤	뜻
precision	해당 클래스라고 예측한 판단의 정확성
recall	실제 해당 클래스를 얼마나 찾아냈는지
f1-score	정밀도와 재현율의 균형
support	시험 데이터에 실제 해당 클래스가 몇 개 있었는지

평균 항목

- **macro avg:** 각 클래스를 같은 중요도로 보고 평균. 소수 클래스 성능도 같은 무게로 반영합니다.
- **weighted avg:** support가 큰 클래스에 더 큰 비중을 주어 평균. 다수 클래스 영향이 큼.

평가 순서

정확도 → 혼동 행렬 → 클래스별 precision-recall-F1 → macro/weighted 평균 순으로 보면 모델의 실수 구조를 놓치기 어렵습니다.

KNN: 가까운 이웃의 다수결

새 데이터 주변에 어떤 클래스가 많은지 보고 분류합니다.

과일	무게	크기
사과 A	180g	8cm
사과 B	170g	7.5cm
귤 A	90g	5cm

새 과일
175g, 7.8cm인 과일의 가까운 이웃 3개가 사과·사과·귤이면 사과 2표로 최종 분류합니다.

```
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
```

K의 크기	특징	위험
너무 작음	아주 가까운 데이터에 민감	이상치 하나에 결과가 흔들릴 수 있음
적절함	주변의 지역적 패턴 반영	검증을 통해 선택 필요
너무 큼	멀리 있는 데이터까지 참여	다수 클래스의 영향이 지나치게 커질 수 있음

학습과 예측의 특징

KNN은 복잡한 규칙을 미리 만들기보다 학습 데이터를 기억합니다. 예측 시 새 데이터와 기존 데이터의 거리를 계산하므로 학습은 단순하지만 데이터가 많을수록 예측 비용이 커질 수 있습니다.

한 문장 핵심
KNN은 새로운 데이터와 가장 가까운 K개의 기존 사례를 찾고 **이웃들의 다수결**로 클래스를 결정합니다.

KNN에서 표준화가 중요한 이유

KNN은 거리 기반이므로 숫자 단위가 큰 조건이 거리를 지배할 수 있습니다.

무게 차이 50g과 크기 차이 2cm를 숫자 50과 2로 비교하면 무게가 훨씬 큰 영향을 줍니다. 무게가 실제로 더 중요해서가 아니라 단위가 크기 때문일 수 있습니다.

```
scaler = StandardScaler()

# 학습 데이터에서만 기준을 학습
X_train = scaler.fit_transform(X_train)

# 같은 기준을 시험·새 데이터에 적용
X_test = scaler.transform(X_test)
new_samples = scaler.transform(new_samples)
```

왜 시험 데이터에 fit하면 안 될까?

시험 데이터의 평균과 분포를 미리 사용하면 시험문제의 정보를 학습 과정에 흘려보내는 셈입니다. 이를 **데이터 누수**라고 합니다.

먼저 분리

→

train으로 fit

→

train 변환

→

test 변환

붓꽃 KNN

```
iris = load_iris()
X, y = iris.data, iris.target

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
predictions = knn.predict(new_samples)
```

노트북의 두 KNN 예제

첫 KNN 예제는 원본 길이를 그대로 사용하고, 뒤의 KNN 예제는 StandardScaler를 사용합니다. 붓꽃 네 특성은 모두 cm 단위라 큰 문제는 없지만, 거리 모델의 일반 원칙을 배우려면 두 번째 흐름이 더 안전합니다.

의사결정나무: 질문을 따라 분류

조건 질문을 순서대로 던지고 답에 따라 가지를 이동해 최종 클래스에 도착합니다.

```

꽃잎 길이가 2.5cm 이하인가?
├─ 예      → Setosa
└─ 아니요 → 꽃잎 너비가 1.7cm 이하인가?
               ├─ 예      → Versicolor
               └─ 아니요 → Virginica
  
```

어떤 질문을 고를까?

모델은 질문 전에는 여러 품종이 섞여 있지만 질문 후 각 그룹에 한 품종이 더 많이 모이도록 하는 기준을 찾습니다. 즉, 클래스를 가장 깔끔하게 나누는 특성과 기준값을 선택합니다.

```

clf = DecisionTreeClassifier(max_depth=3, random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
  
```

설정	뜻	주의
max_depth	질문을 이어갈 최대 단계	너무 깊으면 학습 데이터를 세세하게 외움
min_samples_split	추가 질문으로 나누기 위한 최소 데이터 수	작을수록 복잡한 나무 가능
min_samples_leaf	마지막 잎에 남아야 할 최소 데이터 수	작을수록 희귀 사례까지 따로 외울 수 있음

장점

판단 경로를 시각화하기 쉽고 표준화가 필수는 아닙니다.

단점

깊어지면 과적합되기 쉽고 데이터가 조금 바뀌어도 구조가 달라질 수 있습니다.

한 문장 핵심

의사결정나무는 데이터를 가장 잘 나누는 질문을 반복해 클래스가 비교적 순수한 마지막 그룹을 만듭니다.

랜덤포레스트: 여러 나무의 다수결

한 나무의 불안정성을 줄이기 위해 서로 조금씩 다른 나무를 여러 개 학습합니다.

나무	예측
나무 1	사과
나무 2	사과
나무 3	배
나무 4	사과
나무 5	배

사과 3표, 배 2표이므로 최종 예측은 사과입니다.

나무를 다르게 만드는 방법

- 각 나무가 학습할 사례를 무작위로 다시 뽑습니다. 같은 사례가 중복될 수도 있습니다.
- 질문을 만들 때 고려할 특성 후보도 무작위로 일부 선택합니다.
- 각 나무의 실수가 서로 다르면 다수결에서 잘못된 판단이 상쇄될 수 있습니다.

```
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)
y_pred = rf_model.predict(X_test)
```

의사결정나무	랜덤포레스트
나무 한 개	나무 여러 개
판단 경로 설명이 쉬움	전체 판단 설명은 더 어려움
데이터 변화에 민감	비교적 안정적
과적합되기 쉬움	과적합 위험을 줄일 수 있음

한 문장 핵심

랜덤포레스트는 서로 다른 관점으로 학습한 여러 나무의 의견을 모아 한 나무보다 안정적인 판단을 만듭니다.

GridSearchCV와 교차검증

모델이 스스로 배우는 값이 아니라 사람이 학습 전에 정하는 설정을 하이퍼파라미터라고 합니다.

```
param_grid = {
    "n_estimators": [50, 100, 200],
    "max_depth": [4, 6, 8],
    "min_samples_split": [2, 4],
    "min_samples_leaf": [1, 2]
}

grid_search = GridSearchCV(
    estimator=rf_model,
    param_grid=param_grid,
    cv=3,
    scoring="accuracy"
)
```

cv=3의 동작

회차	학습에 사용	검증에 사용
1회	A, B 묶음	C 묶음
2회	A, C 묶음	B 묶음
3회	B, C 묶음	A 묶음

각 조합을 세 번 평가하고 결과를 종합합니다. 하나의 우연한 분할에서만 잘 나오는 설정보다 여러 분할에서 안정적인 설정을 찾습니다.

```
grid_search.fit(X_train, y_train)
grid_search.best_params_      # 가장 좋은 설정
best_rf_model = grid_search.best_estimator_

# 최종 시험 데이터는 마지막에 한 번 평가
y_pred = best_rf_model.predict(X_test)
```

평가 지표 선택

노트북은 `scoring="accuracy"`를 사용합니다. 클래스가 불균형하거나 소수 클래스를 놓치면 위험한 문제에서는 F1이나 재현율 등 목적에 맞는 지표를 선택해야 합니다.

데이터 역할

하이퍼파라미터 선택은 학습 데이터 내부의 교차검증으로 하고, 시험 데이터는 선택이 끝난 모델의 최종 평가에만 사용합니다.

랜덤포레스트의 특성 중요도

여러 나무가 질문을 만들 때 각 특성이 분류에 얼마나 유용했는지 요약합니다.

특성	중요도 예시	읽는 방법
꽃잎 길이	0.42	분류 질문에서 많이 활용
꽃잎 너비	0.38	분류 질문에서 많이 활용
꽃받침 길이	0.15	상대적으로 덜 활용
꽃받침 너비	0.05	현재 모델에서 활용이 적음

```
feature_importances = best_rf_model.feature_importances_
feature_importance_df = pd.DataFrame({
    "feature": feature_names,
    "importance": feature_importances
}).sort_values("importance", ascending=False)
```

해석할 때 주의

- 중요도가 높다는 것은 현재 데이터와 모델에서 많이 활용했다는 뜻입니다.
- 중요도가 높아도 결과를 일으킨 직접적인 원인이라는 뜻은 아닙니다.
- 비슷한 정보를 가진 두 특성은 중요도가 나뉘어 나타날 수 있습니다.
- 다른 데이터나 설정으로 학습하면 중요도 순서가 달라질 수 있습니다.

회귀계수와 차이

선형 모델의 계수는 조건 변화에 따른 예측 변화 방향과 크기를 나타냅니다. 랜덤포레스트 중요도는 나무들이 해당 특성을 이용해 데이터를 얼마나 유용하게 나눴는지 나타냅니다.

텍스트 분류 전체 흐름

문장을 그대로 모델에 넣을 수 없으므로 숫자 벡터로 변환한 뒤 분류합니다.

메일 문장

→

TF-IDF

→

숫자 벡터

→

분류 모델

스팸 데이터 전처리

```
data = pd.read_csv("./dataset/spam.csv", encoding="ISO-8859-1")
data = data[["v1", "v2"]]
data.columns = ["label", "text"]

data["label"] = data["label"].map({"ham": 0, "spam": 1})
data.dropna(inplace=True)
```

처리	의미
v1, v2 선택	레이블과 본문에 필요한 열만 사용
열 이름 변경	label, text로 역할을 명확하게 표현
ham/spam 매핑	정상 0, 스팸 1의 숫자 레이블로 변경
dropna	레이블이나 본문이 비어 있는 행 제거

encoding

파일이 UTF-8이 아니라 서유럽 문자 인코딩으로 저장돼 있을 수 있어 ISO-8859-1을 지정합니다. 잘못된 인코딩을 쓰면 문자가 깨지거나 읽기에 실패합니다.

클래스 비율

스팸 데이터는 정상 메일이 더 많은 경우가 많습니다. 분할 시 `stratify=data["label"]`을 추가하고 정확도 외에 스팸 클래스의 정밀도·재현율을 확인하는 편이 안전합니다.

TF-IDF: 중요한 단어를 숫자로 표현

특정 문서에서 자주 나오면서 다른 문서에서는 흔하지 않은 단어에 더 큰 값을 줍니다.

메일	무료	당첨	회의	일정
무료 상품 당첨	1	1	0	0
무료 쿠폰 제공	1	0	0	0
회의 일정 확인	0	0	1	1

단순 횟수에서 더 나아가 'the', 'is', 'and'처럼 모든 문서에서 흔한 단어의 중요도는 낮추고 '당첨', '무료', '회의'처럼 구분에 도움이 되는 단어를 강조합니다.

```
vectorizer = TfidfVectorizer(stop_words="english")

X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

sample_tfidf = vectorizer.transform(sample_text)
```

- fit: 학습 메일에서 어휘와 중요도 기준을 학습
- transform: 문장을 같은 기준의 숫자 벡터로 변환
- stop_words="english": 분류에 도움이 적은 흔한 영어 단어 제거

데이터 누수 방지

시험 메일까지 fit_transform하면 시험 데이터의 단어 정보가 학습 기준에 섞입니다. 시험과 새 문장은 반드시 학습 데이터로 만든 vectorizer의 transform만 사용합니다.

나이브 베이즈: 단어를 증거로 판단

각 단어가 스팸과 정상 메일에서 얼마나 자주 등장했는지 학습합니다.

스팸에서 흔한 단어

무료, 당첨, 클릭, 상금, 축하합니다

정상에서 흔한 단어

회의, 일정, 보고서, 프로젝트, 첨부

새 메일

“축하합니다. 무료 상품에 당첨되었습니다. 지금 링크를 클릭하세요.”에는 스팸에서 흔한 증거가 많이 포함되어 스팸으로 분류될 가능성이 높습니다.

왜 Naive, 즉 순진한가?

현실에서는 무료·상품·당첨이 서로 관련될 수 있지만, 계산을 단순하게 하기 위해 각 단어를 독립적인 증거처럼 다룹니다. 완벽한 가정은 아니지만 조건 수가 많은 텍스트에서 빠르고 효과적일 수 있습니다.

```
model = MultinomialNB()
model.fit(X_train_tfidf, y_train)

y_pred = model.predict(X_test_tfidf)
predictions = model.predict(sample_tfidf)
```

장점	주의점
학습·예측이 빠르고 텍스트 빈도 정보에 잘 맞춤	단어들이 독립적이라는 단순한 가정을 사용
조건이 매우 많은 희소 벡터 처리에 적합	문맥과 단어 순서를 직접 이해하는 모델은 아님

한 문장 핵심

나이브 베이즈는 새 문장의 단어들이 어느 클래스에서 더 자연스럽게 등장할 조합인지 판단합니다.

SVM: 가장 여유 있는 경계 찾기

두 클래스 사이에 여러 경계가 가능할 때 양쪽 데이터에서 최대한 멀리 떨어진 경계를 선택합니다.

과일 예시

사과 그룹과 귤 그룹 사이의 가운데에 넓은 여유 공간을 두는 경계를 찾으면 새 데이터가 조금 흔들려도 분류가 쉽게 바뀌지 않습니다.

마진과 Support Vector

- **마진:** 경계와 양쪽 클래스 사이의 여유 공간
- **Support Vector:** 경계에 가장 가까워 경계 위치를 결정하는 핵심 데이터

```
model = SVC(kernel="linear", random_state=42)
model.fit(X_train_tfidf, y_train)
y_pred = model.predict(X_test_tfidf)
```

kernel="linear"는 직선·평면·더 높은 차원의 평평한 경계를 사용한다는 뜻입니다. TF-IDF처럼 특성이 매우 많은 텍스트 데이터에서도 사용할 수 있습니다.

장점	주의점
클래스 사이의 안정적인 경계를 찾음	일반 숫자 데이터에서는 표준화가 중요
특성이 많은 텍스트 데이터에 활용 가능	데이터가 매우 크면 학습이 느릴 수 있음
경계 근처 핵심 데이터에 집중	기본 SVC는 확률을 직접 제공하지 않음

커널의 역할

선형 경계로 나누기 어려운 데이터에는 곡선 경계를 만들 수 있는 다른 커널도 있습니다. 다만 노트북은 텍스트 분류에 linear를 사용합니다.

분류 모델 한눈에 비교

모델	핵심 판단 방식	표준화	설명 용이성
로지스틱 회귀	가능성과 임계값	권장	비교적 쉬움
KNN	가까운 이웃 다수결	중요	쉬움
의사결정나무	조건 질문	필수 아님	매우 쉬움
랜덤포레스트	여러 나무 다수결	필수 아님	보통
나이브 베이즈	클래스별 특징 출현	상황에 따라 다름	보통
SVM	가장 여유 있는 경계	중요	어려운 편

상황별 출발점

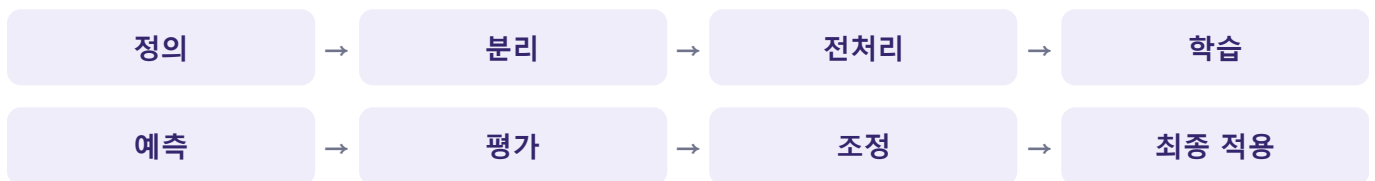
- 가능성과 조건 방향을 보고 싶다: 로지스틱 회귀
- 비슷한 사례끼리 모이고 데이터가 적다: KNN
- 판단 근거를 질문 형태로 설명해야 한다: 의사결정나무
- 표 데이터에서 안정적인 기본 성능이 필요하다: 랜덤포레스트
- 빠른 텍스트 분류 기준 모델이 필요하다: 나이브 베이즈
- 특성이 많은 데이터에서 분명한 경계를 찾고 싶다: SVM

최고의 모델은 미리 정해지지 않음

데이터 크기, 클래스 불균형, 설명 가능성, 예측 속도, 어떤 실수가 더 위험한지에 따라 선택이 달라집니다. 같은 분할과 같은 평가 기준으로 여러 모델을 비교해야 합니다.

분류 프로젝트 전체 흐름

1. **문제 정의:** 어떤 레이블을 예측할지, 어떤 실수가 더 위험한지 정합니다.
2. **X와 y 준비:** 입력 특징과 실제 클래스 레이블을 분리합니다.
3. **데이터 확인:** 결측치, 클래스 개수와 비율, 이상치를 확인합니다.
4. **학습·시험 분리:** 필요하면 stratify=y로 클래스 비율을 유지합니다.
5. **전처리:** 표준화나 TF-IDF를 학습 데이터에서만 fit합니다.
6. **모델 학습:** fit(X_train, y_train)을 실행합니다.
7. **예측:** predict(X_test)로 시험 레이블을 구합니다.
8. **평가:** 정확도, 혼동 행렬, 정밀도, 재현율, F1을 확인합니다.
9. **설정 선택:** 필요하면 학습 데이터 내부의 교차검증을 사용합니다.
10. **새 데이터:** 학습 때 만든 동일한 전처리 기준을 적용한 뒤 예측합니다.



반드시 피할 데이터 누수

```

# 올바른 예
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 잘못된 예
scaler.fit(X_test) # 시험 정보를 미리 봄
  
```

한 문장 핵심

시험 데이터는 모델과 전처리 기준을 선택하는 데 사용하지 않고, 모든 선택이 끝난 뒤 처음 보는 문제처럼 평가합니다.

현재 노트북 실행 전 확인사항

개념과 별개로 현재 프로젝트 상태에서는 일부 셀이 그대로 실행되지 않습니다.

항목	현재 상태	필요한 조치
LogisticRegressionData.csv	C:\work\dataset에 없음	파일 추가 또는 올바른 경로·데이터로 수정
spam.csv	C:\work\dataset에 없음	파일 추가 후 v1, v2 열 존재 확인
데이터 경로	./dataset와 ../dataset 혼용	노트북 실행 위치를 기준으로 경로 통일
reshape 셀 50	주석과 코드가 다름	p.reshape(3, -1)로 의도 명확화
첫 KNN 예제	표준화 없음	일반 원칙 학습에는 StandardScaler 흐름 권장
분류 데이터 분할	일부 예제에 stratify 없음	클래스 비율 유지가 필요하다면 stratify=y 추가

경로가 달라지는 이유

- 작업 폴더가 C:\work이면 ./dataset/file.csv가 맞습니다.
- 작업 폴더가 C:\work\supervisedLearning이면 ../dataset/file.csv가 맞습니다.
- os.getcwd()로 현재 작업 폴더를 확인한 뒤 한 방식으로 통일해야 합니다.

중요

CSV 파일이 없어도 scikit-learn에 내장된 붓꽃 예제는 실행할 수 있습니다. 로지스틱 회귀와 스팸 예제는 필요한 외부 데이터 파일을 먼저 준비해야 합니다.

표 데이터 분류 코드 빠른 참조

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report
)

# 1. 분리: 클래스 비율 유지
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 2. 전처리: train에서만 기준 학습
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 3. 모델 학습과 예측
model = LogisticRegression()
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)

# 4. 평가
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion matrix:\n", confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

# 5. 새로운 데이터도 같은 기준으로 변환
new_scaled = scaler.transform(new_data)
print(model.predict(new_scaled))
print(model.predict_proba(new_scaled))
```

모델 교체

같은 전처리·평가 구조에서 LogisticRegression을 KNN, 의사결정나무, 랜덤포레스트, SVM 등으로 바꾸어 비교할 수 있습니다. 단, 모델마다 표준화 필요성과 제공 기능이 다릅니다.

최종 복습 체크리스트

- ✓ 회귀와 분류를 y 의 성격으로 구분할 수 있다.
- ✓ 이진 분류와 다중 분류의 차이를 설명할 수 있다.
- ✓ 실제 레이블, 클래스 가능성, 최종 예측을 구분할 수 있다.
- ✓ 임계값을 바꾸면 정밀도와 재현율이 달라지는 이유를 안다.
- ✓ 정확도만으로 불균형 데이터를 평가하면 위험한 이유를 안다.
- ✓ TP, TN, FP, FN을 질병 예시로 설명할 수 있다.
- ✓ 정밀도, 재현율, F1, support의 의미를 구분할 수 있다.
- ✓ KNN의 K와 표준화 필요성을 설명할 수 있다.
- ✓ 의사결정나무와 랜덤포레스트의 차이를 설명할 수 있다.
- ✓ GridSearchCV가 학습 데이터 내부에서 설정을 비교하는 이유를 안다.
- ✓ TF-IDF가 문장을 숫자로 바꾸는 이유를 설명할 수 있다.
- ✓ 나이브 베이즈와 SVM의 판단 방식을 구분할 수 있다.
- ✓ 전처리 기준을 시험 데이터에 fit하면 안 되는 이유를 안다.

모델별 한 문장

모델	기억할 문장
로지스틱 회귀	가능성을 구해 임계값으로 레이블 선택
KNN	가까운 이웃들의 다수결
의사결정나무	조건 질문을 따라 마지막 클래스 도착
랜덤포레스트	서로 다른 여러 나무의 다수결
나이브 베이즈	특징이 클래스별로 등장하는 경향을 증거로 사용
SVM	클래스 사이의 가장 여유 있는 경계 탐색

최종 한 문장

분류는 과거의 입력 x 와 정답 레이블 y 를 학습하여, 새로운 입력이 어느 클래스에 속하는지 결정하는 지도학습 방법입니다.